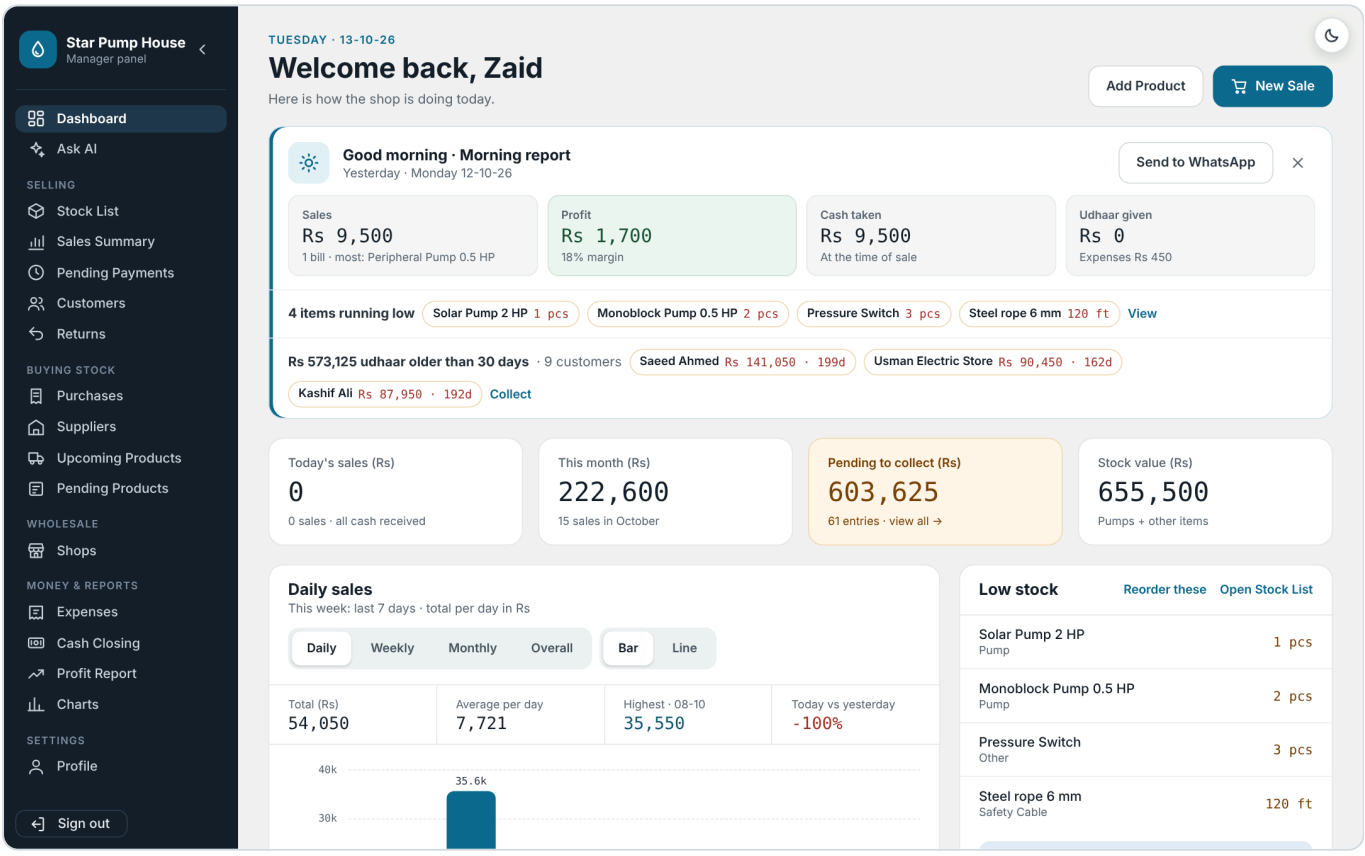
CASE STUDY · ZAID MUZAFFAR · 2026

Star Pump House Shop Management System

How I replaced the paper registers of my water-pump and hardware shop with a web app that runs the whole business: stock, sales, credit, suppliers, wholesale accounts, cash, profit and an AI assistant.

ROLE	PLATFORM	STACK	STATUS
Solo: product, UX, code, deployment	Web app, desktop and phone	JavaScript · Firebase · Gemini	Live, in daily use



The Dashboard on a normal shop day (demo data).

LIVE DEMO
starpumphouse.web.app

DEMO LOGIN
demo@starpumphouse.com · Star123@

CODE
github.com/zaidmuzaffar/star-pump-house

SUMMARY

00 At a glance

Star Pump House sells water pumps, electric wire, safety cable and hardware, and also supplies other shops on account. All of this used to be written by hand. I designed and built a web app that the shop now uses every day, from the counter on a phone and from a laptop in the evening.

23
screens: selling, buying, wholesale, money and settings

57
releases shipped from real feedback in the shop

17
data tools the AI assistant can use to answer questions

Rs 0
monthly running cost on Firebase's free plan

The challenge

Stock, sales, credit (*udhaar*), supplier bills and wholesale accounts lived in separate notebooks. Nobody could say quickly what was running low, who owed what, or what the shop really earned.

The result

One system where every sale updates stock, cash and credit together. The owner sees the whole business on one screen, closes the cash drawer in minutes, and can ask questions in plain English or Roman Urdu.

Contents

1. Background and the problem
2. Goals and constraints
3. Users and their daily jobs
4. Process: how it was built
5. Architecture and technology choices
6. Data model
7. The product, module by module
8. The AI assistant
9. Design decisions
0. Technical challenges and how I solved them
1. Security, privacy and reliability
2. Testing and quality
3. Results and impact
4. What I learned and what is next

CONTEXT

01 Background and the problem

Star Pump House is a small shop with a mix of retail and wholesale work. On a normal day it sells pumps by the piece, wire and cable by the foot, and small hardware like foot valves and pressure switches. Some customers pay cash; many regular customers and plumbers take goods on credit (*udhaar*) and pay later. The shop also buys from several suppliers on account and gives stock to other shops that pay over time.

How it worked before

- Sales and credit were written in a register, page by page.
- Stock was counted by eye; wire was guessed from how full a bundle looked.
- Supplier bills and payments were kept in a separate notebook.
- Wholesale shops had their own khata (account book).

- At night, cash in the drawer (*galla*) was counted with no clear "expected" figure.
- Profit was a feeling, not a number: costs and expenses were never subtracted properly.
- Finding one customer's history meant flipping through months of pages.

The real problems behind it

Problem	What it caused
No live stock level	Popular pumps ran out before re-ordering; wire bundles were over-sold.
Credit spread across pages	Old udhaar was forgotten; customers disputed amounts with nothing to show them.
No link between sales, cash and credit	The cash drawer rarely matched, and nobody knew why.
Supplier and wholesale accounts by hand	Balances drifted; bills were paid twice or missed.
No profit figure	Pricing and buying decisions were made without data.

The brief: open one screen in the morning and know how yesterday went, what is running low, and who owes the shop money.

FRAMING

02 Goals and constraints

Goals

1. Record a sale in seconds, on a phone, at the counter.
2. Keep stock, cash and credit always consistent with each other.
3. Show who owes what and for how long.
4. Track suppliers, purchases and wholesale shops in the same place.
5. Give a true profit figure after costs and expenses.
6. Make the daily cash count quick and exact.

Constraints

- **Zero budget:** no paid servers or subscriptions.
- **Non-technical users:** the people at the counter are not used to software.
- **Phones first:** most work happens on a phone, sometimes with weak internet.
- **Mixed units:** pieces for pumps, feet and bundles for wire.
- **Language:** the shop thinks in Urdu terms (udhaar, galla, khata).
- **Data must never be lost:** the records are the business.

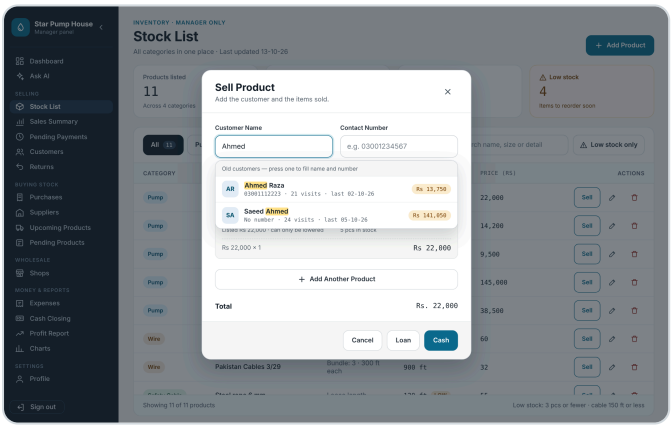
PEOPLE

03 Users and their daily jobs

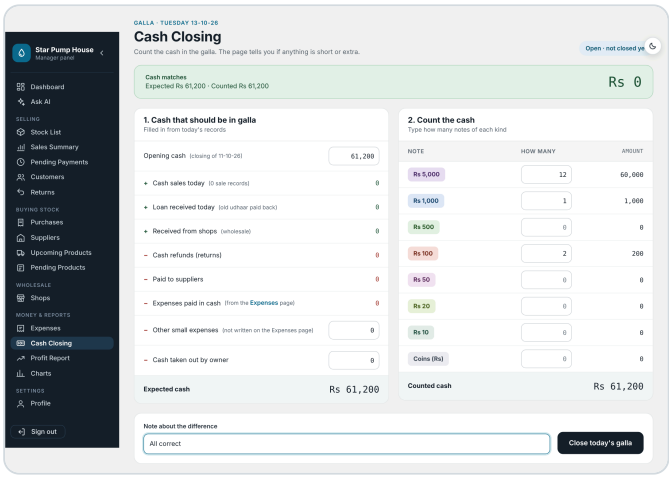
User	What they need to do	What the app gives them
Owner / manager	See the whole business, sell, collect credit, buy stock, pay suppliers, close the day, check profit.	Every module, the morning report, reports, charts, the AI assistant, backups and staff control.
Salesman	Sell from the counter without seeing money, profit or accounts.	A restricted login: Stock List and Sell only. Their sales are tagged with their name.
Customer (indirect)	Know what they bought and what they still owe.	Receipts as PDF or WhatsApp text, and a printable statement.
Wholesale shop (indirect)	Settle their account with the shop.	A running khata with bills, payments, returns and a "Total Bill" PDF.

The day in the shop, mapped to the app

Morning report → Sell (cash or loan) → Collect udhaar → Receive stock → Record expenses → Cash closing → Profit & charts



Selling: old customers are suggested as you type; several products go on one bill.



Closing the day: the app works out the expected cash; the owner counts the notes.

APPROACH

04 Process: how it was built

I built the app in small steps and put each one into daily use straight away. Real use in the shop decided what came next: every complaint, slow screen or confusing word became the next change. In total that meant **57 releases**, each shipped as a small, tested update.

- **Phase 1 · Stock list**
  Products in four categories (pump, wire, safety cable, other), with feet and bundles for wire and a LOW badge.
- **Phase 2 · Selling and credit**
  Multi-item bills, Cash or Loan, partial payments, Pending Payments and customer accounts built automatically from sales.
- **Phase 3 · Buying stock**
  Suppliers, purchase bills, payments, and a pipeline from Pending (planned) to Upcoming (ordered) to Arrived (in stock).
- **Phase 4 · Wholesale and returns**
  Accounts for other shops, customer and shop returns, damaged goods kept out of stock.
- **Phase 5 · Money and reports**
  Expenses, Cash Closing, Profit Report, Charts, receipts and statements as PDF, the morning report.
- **Phase 6 · AI assistant**
  Questions answered from the records, and entries made through pre-filled forms; voice input in Urdu or English.
- **Phase 7 · Hardening**
  Salesman logins, App Check, one login per browser, backups, speed work and many small usability fixes.

How feedback turned into changes (examples)

What the shop said	What I changed
"Daily sales only shows a few days."	Clear period rules: Daily = last 7 days, Weekly = last 30 days grouped by week, Monthly = 12 months.
"Pages are slow; headings show first, numbers later."	Records cached on the device, links preloaded, the next page prepared in advance.
"I signed out by mistake."	A confirmation dialog before signing out.
"The error after sign-up makes no sense to a salesman."	Plain messages, and "this email is already used" is checked before password rules.
"A light touch on a menu item left it highlighted."	Hover effects switched off on touch screens.
"The mic does not open on the iPhone."	Microphone permission requested properly, with clear guidance when it is blocked.

ENGINEERING

05 Architecture and technology choices

Browser: 23 HTML pages + ES modules → Firebase Auth → Cloud Firestore (cached on device)

Ask AI page → Firebase AI Logic (Gemini) ⇄ 17 tools that read records in the browser

App Check (reCAPTCHA Enterprise) → only the real site may reach the database and the AI

Firebase Hosting → free HTTPS site, one command to deploy

Decision	Why
Plain HTML, CSS and JavaScript modules, no framework	No build step to break, tiny pages that load fast on cheap phones, and every file can be read and fixed directly.
Firebase (Auth, Firestore, Hosting)	Free for a shop this size, managed security, real-time sync between phone and laptop, nothing to run myself.
Firestore persistent cache	Pages open with data instantly and keep working on weak internet; it syncs in the background.
Firebase AI Logic with Gemini	No API key in the code or own server; works with App Check; supports function calling.
PDFs made in the browser	A small built-in PDF writer for receipts and statements, so no library or server is needed.
Versioned file URLs	Every update reaches every phone on the next reload, without stale cached files.

Code at a glance

About 8,500 lines of JavaScript and 1,200 lines of CSS across 23 pages. Shared code lives in a few modules: login and access checks, number and date helpers, dialogs, PDF writing, the sidebar and the theme.

ENGINEERING

06 Data model

Each owner's records sit under `users/{uid}/`, so one owner can never reach another's data. A bill with several products is stored as one record per line, linked by a shared timestamp, which keeps per-item profit and returns simple.

Collection	Holds	Feeds
products	category, name, size, quantity or feet, bundles, price, cost	Stock, Sell, stock value
sales	item, quantity/feet, amount, paid at sale, cost, customer, salesman, date	Sales, credit, customers, profit, charts, closing

Collection	Holds	Feeds
payments	credit received later, linked to bills	Pending Payments, closing
returns	customer or shop, refund / cut credit / exchange, back to stock or damaged	Returns, profit
suppliers · purchases · supplierPayments	supplier details, bills with items, payments by method	Purchases, supplier accounts
shops · shopBills · shopPayments	wholesale shops, stock given, payments	Wholesale accounts
upcoming · pendingProducts	ordered and planned stock	Buying pipeline
expenses · closings · settings	running costs, daily cash counts, shop name and backup date	Profit, closing, morning report

Key formulas. Credit left on a bill = amount − paid at sale − later payments − credit cut by returns. Expected cash = opening + cash sales + credit received + shop payments − refunds − supplier payments − cash expenses − owner withdrawals. Net profit = (sales − cost of items sold, after returns) − expenses.

FEATURES

07 The product, module by module

Dashboard

Morning report for the last working day (sales, profit, cash, credit given and received, expenses), low stock, credit older than 30 days, a weekly backup reminder, KPIs and a sales chart.

Stock List

Four categories with their own fields. Wire and cable kept in feet with bundles that update as feet are sold. LOW badge at 3 pieces or 150 ft.

Sell

Multi-item bills, old-customer suggestions, price can only go down, Cash with amount received or Loan; the unpaid part becomes credit.

Sales Summary

Today, 7 days, 30 days, 12 months or overall. Same customer + same day merged into one row. Edit customer, change Cash/Loan, receipt PDF, WhatsApp.

Pending Payments

Every open credit with days outstanding; full or partial collection; bulk actions.

Customers

Built automatically from sales by phone or name; purchase history, payments, returns and a statement PDF.

Returns

From customers (refund, cut credit, exchange) or wholesale shops; damaged goods kept aside instead of returning to stock.

Purchases & Suppliers

Supplier bills with items, payments by cash, bank, cheque or mobile wallet, and each supplier's running balance.

Upcoming & Pending Products

Planned → ordered → arrived. "Arrived" adds the stock or creates the product.

Wholesale Shops

Give stock on account, receive payments and returns, running khata and Total Bill PDF.

Expenses

Rent, bills, salary, tea, transport, repairs; cash expenses flow into Cash Closing and all expenses into net profit.

Cash Closing

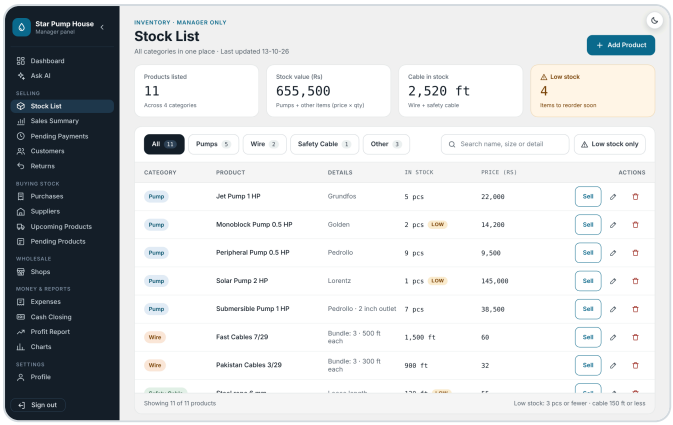
Expected cash worked out line by line; count notes by denomination; Matches / Short / Extra; tomorrow opens with today's count.

Profit Report & Charts

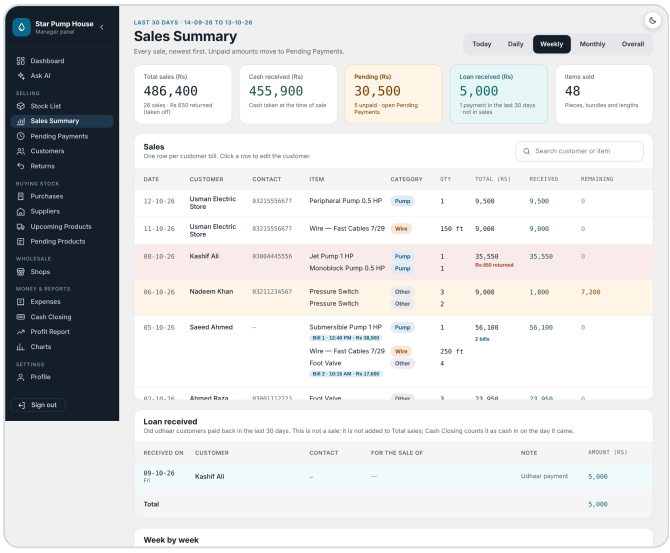
Gross and net profit by item and category; month-by-month, last 30 days, top items, categories and weekdays.

Profile & Settings

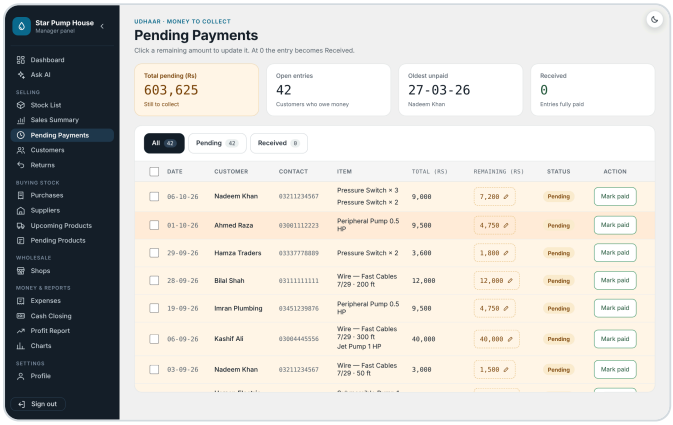
Shop name and phone, password, email verification, salesman logins, JSON backup and restore.



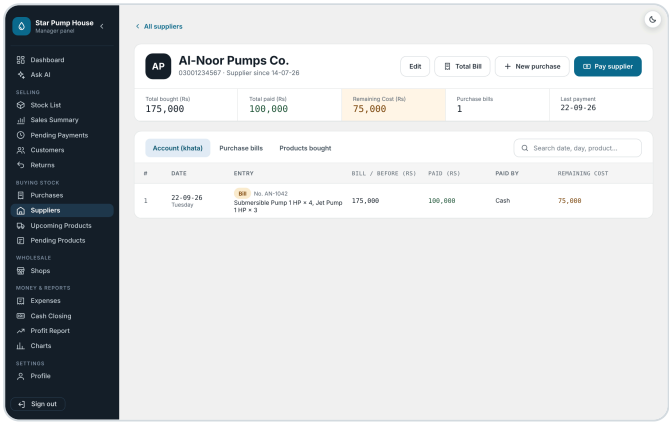
Stock List with categories, LOW badges and quick actions.



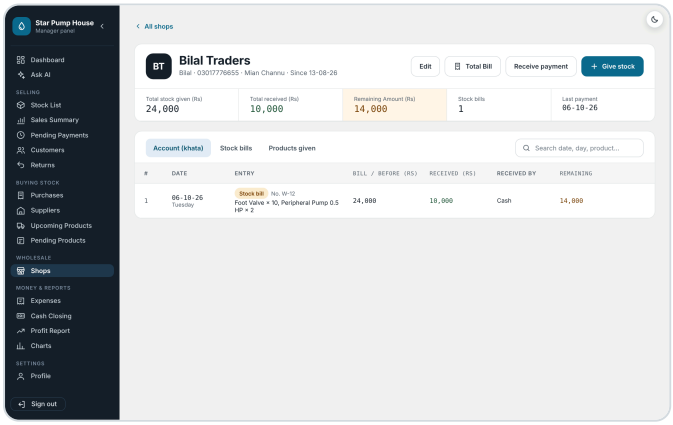
Sales Summary: period buttons, KPIs and one row per customer bill.



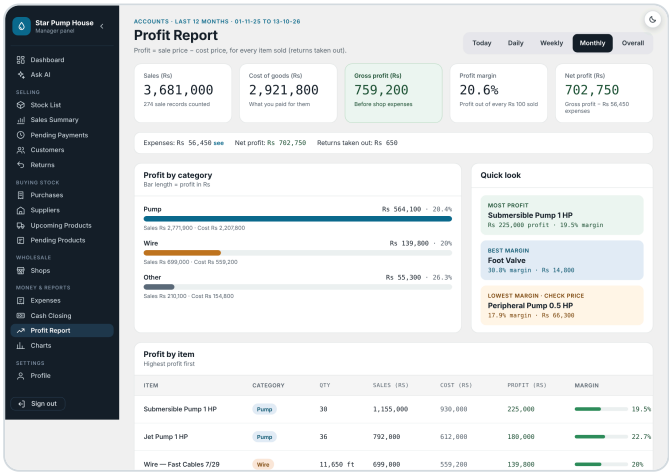
Pending Payments: every open credit and how long it has been open.



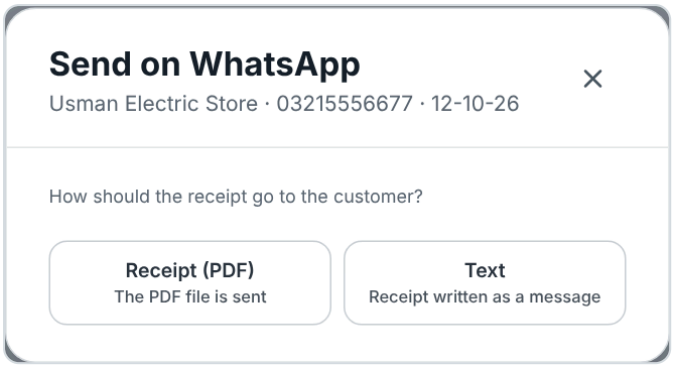
A supplier account: bills, payments and balance.



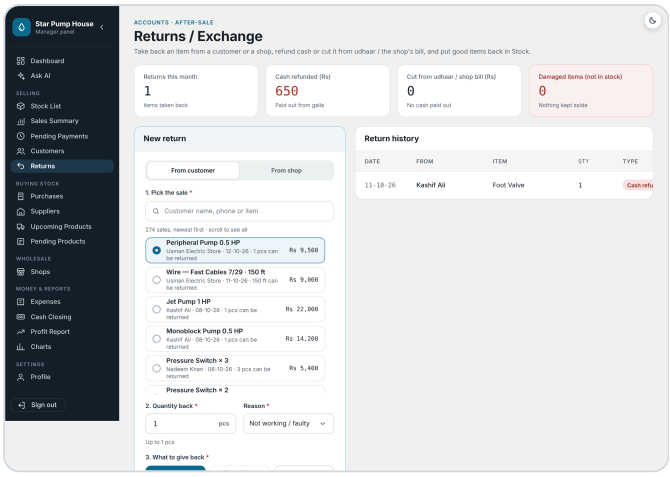
A wholesale shop's khata.



Profit Report: gross and net profit by item.



Receipt on WhatsApp: PDF file or text message.



Returns from customers and shops.

FEATURES

08 The AI assistant

The Ask AI page lets the owner talk to the shop's records, by typing or by voice, in English or Roman Urdu: "how much udhaar does Kashif have?", "which pumps sold most this month?", "aaj bijli ka bill 8500 diya".

How it works

Question → Gemini picks a tool → Tool reads records in the browser → Gemini writes the answer → Form opens if it is an entry

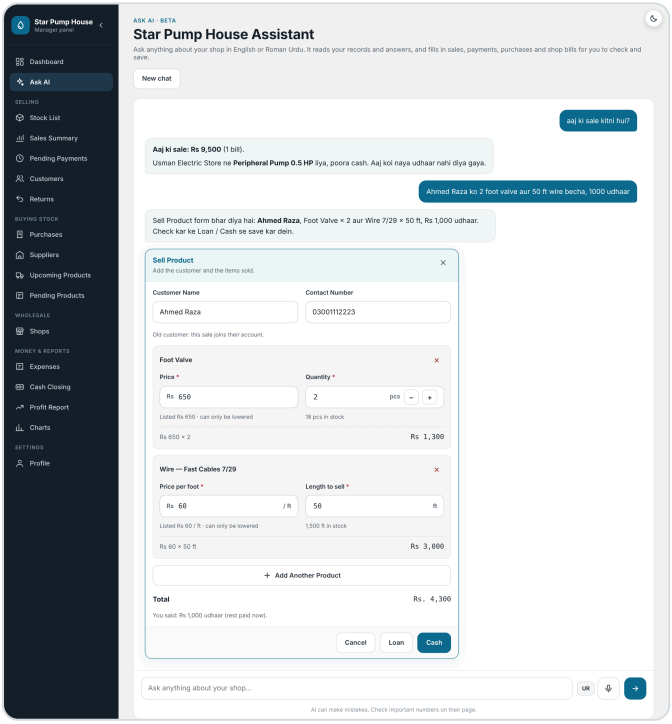
Reads through 17 tools

Shop overview, find sales, customer account, credit list, stock lookup, best sellers, sales by period, profit, expenses, supplier and shop accounts, purchases, returns, orders, daily cash, and stock and sales forecasts. The model never receives the whole database, only what a tool returns.

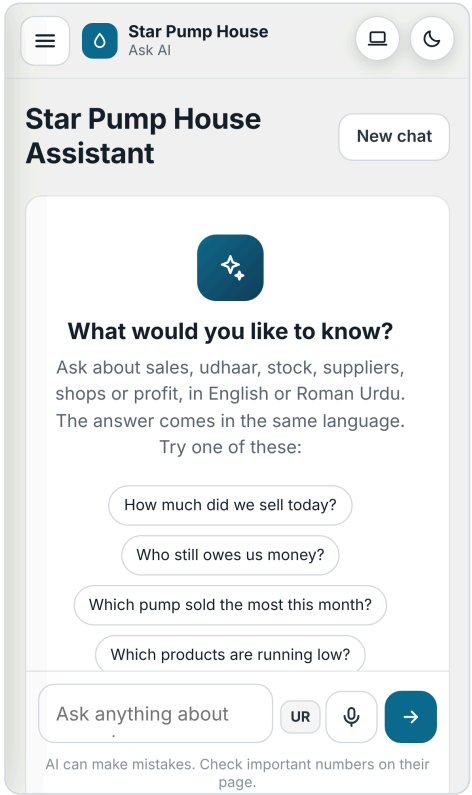
Never saves on its own

For entries (a sale, a payment, a purchase, an expense, a return, a new product, an edit or a delete) it opens the same form the normal page uses, already filled in. The owner checks it and presses Save. If a name matches several records, it shows buttons to choose from.

Built for the free tier. It moves between several Gemini models when one is busy, waits and retries on rate limits, and is protected by App Check so no other site can use the quota.



An answer from the records, then a filled-in Sell form to confirm.



Ask AI on a phone, with the mic for Urdu or English.

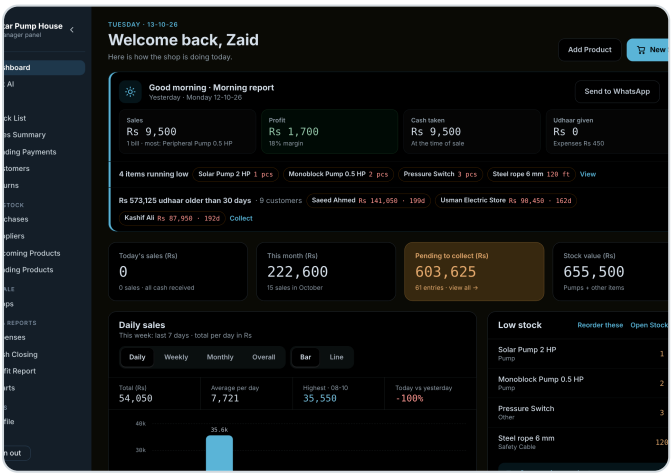
UX

09 Design decisions

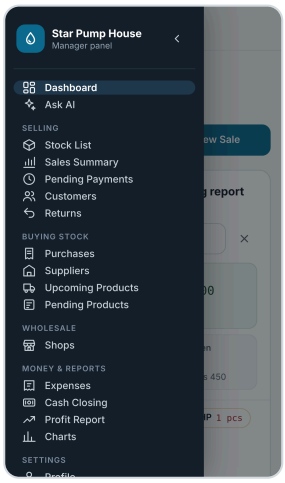
Decision	Reason
Cash or Loan in one tap at the end of a sale	The two real outcomes at the counter; anything unpaid becomes credit automatically.
Price can only be lowered while selling	Allows a discount, prevents accidental over-charging.
Same customer, same day = one row	Matches how the shop thinks about a visit, while each bill stays separate inside.
Wire in feet, bundles automatic	Staff sell in feet; bundles are worked out so nobody has to calculate them.
Colours carry meaning	Amber = credit/pending, green = received, red = returns or short cash, used the same everywhere.
Local words where they help	Udhaar, galla and khata are the words the shop uses; the rest of the interface is simple English.
Phone layout with a slide-in menu	Big touch targets, no hover-only actions, a "Laptop view" option when a wide table is needed.
Confirm before anything risky	Deletes and sign-out ask first; mistakes like a wrong Cash/Loan can be corrected later.
Plain error messages	Technical errors are translated into what the person should do next.
Dark mode	For evening work and personal preference, on every page.

Visual language

A calm, work-focused palette: primary blue **#0E6A8C**, a dark menu **#14202B** and a light paper-like background. IBM Plex Sans for text and IBM Plex Mono for numbers, so columns of amounts line up and are easy to scan.



Dark mode.



Phone menu grouped the way the shop works.

ENGINEERING

10 Technical challenges and how I solved them

Challenge	Solution
Keeping stock, cash and credit consistent when a sale, a payment, a return or a correction can each change several numbers.	Every figure is derived from the underlying records with shared helper functions (remaining credit, cash at sale, returns applied), instead of storing totals that can drift.
Slow page changes and numbers appearing after the headings.	Firestore persistent cache across tabs, cached login role, module preloading and prerendering the likely next page.
Old files on phones after an update, and a crash when two versions of the same module loaded together.	One version number for every file URL, bumped together on each release, plus no-cache headers for HTML, JS and CSS.
Wire sold in feet but stored in bundles.	Feet are the source of truth; bundles are recalculated from feet per bundle, so the count drops on its own.
Owner vs salesman access.	A staff index maps a salesman's email to the owner; security rules let a salesman read stock, create sales and only lower quantities.
One login per browser.	A login event in local storage tells other tabs of the same browser to sign out; other devices are not affected.
Errors while signing out, as live listeners closed.	A signing-out flag silences the expected errors; real loading errors still show.
Free AI limits.	Model rotation, back-off on rate limits, short answers, and App Check to block outside use.
Voice input on iPhone.	Explicit microphone permission request with retry and clear guidance when it is blocked.

TRUST

11 Security, privacy and reliability

Security

- Login required on every page; the page stays hidden until it is confirmed.
- Firestore rules: each owner reaches only their own records.
- Role-based access for salesmen, enforced in the rules, not only in the screens.
- App Check monitored first, then enforced step by step once traffic looked clean.

Privacy and reliability

- The session ends when the browser closes; sign-out wipes the device cache.
- One-click JSON backup of every collection, and restore from file.
- A weekly backup reminder on the Dashboard.
- Backup files kept out of the website and out of the public code.
- A separate demo account with made-up data for anyone who wants to try it.

QUALITY

12 Testing and quality

- **Automated browser tests** with Playwright, using stand-in versions of Firebase so every page can be loaded with realistic demo data and checked without touching real records.
- **Bugs reproduced before they are fixed:** for example, the sign-out error was first recreated in a test, then fixed, then re-tested.
- **Phone and desktop checks** at iPhone size and laptop size, in light and dark mode, for every page.
- **Real-world testing** in the shop after every release, with problems reported from the counter and fixed in the next one.
- **Careful releases:** each update lists exactly which files changed and how to apply them, and is deployed with one command.

OUTCOME

13 Results and impact

Daily	Exact	Rs 0
used in the shop instead of paper registers	expected cash at closing, worked out automatically	hosting and running cost

- **One source of truth:** stock, sales, credit, suppliers and wholesale accounts all agree with each other.
- **Credit under control:** every open amount, its age and the customer's full history are one tap away, with a statement to show them.
- **Real profit:** profit after costs and expenses, by item and by month.
- **Faster counter work:** repeat customers are suggested, receipts go out on WhatsApp, and a salesman can sell without seeing the accounts.
- **Better buying:** low stock and best sellers are visible every morning, so re-orders can happen before items run out.

REFLECTION

14 What I learned and what is next

What I learned

- Modelling a real business in a NoSQL database, and deriving numbers instead of storing totals.
- Authentication, security rules, roles and App Check working together.
- Designing an AI feature that is useful and safe: tools to read, forms to write, a human always confirming.
- Performance on cheap phones: caching, preloading and keeping pages small.
- Shipping small, frequent releases and listening to the people who use them.

What is next

- WhatsApp reminders for old udhaar.
- Barcode / QR scanning for faster selling.
- A custom domain (starpumphouse.com).
- Purchase suggestions based on sales speed and lead time.
- Multi-branch support for more than one shop.

TRY IT

The live demo has six months of made-up sales, customers and suppliers, so every page has something to show.

LIVE DEMO
starpumphouse.web.app

LOGIN
demo@starpumphouse.com · Star123@

CODE
github.com/zaidmuzaffar/star-pump-house

All screenshots use demo data; no real customer information is shown.